

Reinforcement Learning for Bomberman

Luis David Maco Cangalaya Diana Krasnokutskaya Daniela Ardila Porras

Machine Learning Essentials – Summer Semester 2026

1 Introduction

Author: Diana Krasnokutskaya

Bomberman is a simple game with a surprisingly hard decision problem underneath. In the version used for this project, four agents move on a 17×17 grid of walls, crates and free tiles for at most 400 steps. In each step an agent picks one of six actions: four moves, waiting, or dropping a bomb. A bomb explodes after four steps and destroys everything within three tiles in a straight line until a wall stops the blast. Crates can hide coins. A coin is worth one point, and blowing up an opponent is worth five. The agent has 0.5 seconds of CPU time per step to decide what to do. Our goal was to train an agent with reinforcement learning (RL) that plays this game as well as possible, and in particular one that beats the provided `rule_based_agent` on the `classic` board.

We worked through four tasks of increasing difficulty:

- (1) collecting coins on an empty board
- (2) destroying crates and finding hidden coins without killing oneself
- (3) hunting the provided `peaceful_agent` and `coin_collector_agent`
- (4) holding one's own against the full-strength `rule_based_agent`.

Tasks 1–3 serve as stepping stones and debugging stages. Task 4 is the criterion the tournament actually rewards.

The task is small and cheap to simulate, but still includes many of the challenges that make reinforcement learning difficult in practice.

- **Delayed and sparse credit.** Rewards are rare. An agent that walks into its own blast dies four steps after the decision that actually killed it, and the step that traps an opponent happens well before the opponent dies. Credit has to travel back through several steps.
- **Safety-critical actions.** One wrong bomb ends the round. For every agent we trained, a large share of all losses came from suicides rather than from opponents.
- **A non-stationary, multi-agent environment.** From task 3 on, other agents drop bombs and compete for the same coins, so a safe state can become deadly because of someone else's decision.
- **A large state space.** Tabular Q-learning over raw boards is impossible. We either design a compact feature representation by hand or let a neural network learn one.
- **Limited compute and time.** Three weeks, one CPU thread per agent in the tournament.

We developed two structurally different agents, `q_agent` and `dqn_agent`, which approach the learning problem in different ways. While `q_agent` relies on manually designed representations incorporating domain knowledge, `dqn_agent` learns its representation directly from the game state. This allows us to investigate the central question of this report: under a fixed time budget, how does the use of domain knowledge compare to learning representations directly from the environment?

After the final frozen head-to-head evaluation, we submitted `q_agent`. It achieved the higher score in the paired tournament-shaped comparison, although the DQN remained safer in terms of suicide rate.

2 Background

Author: Daniela Ardila

2.1 Bomberman as a Markov decision process

Building on the rules summarized in Section 1, we describe here the details of the game that are relevant to decide which method is the best approach, and state the game in the notation of the lecture [5]. An explosion stays lethal for one more step after the detonation, and each agent can only have one bomb at a time. Only the wall layout is fixed. Crates, coins and start corners are re-randomized every round, and on the `classic` board all nine coins start hidden under crates. So the agent cannot memorize a board, but has to generalize to new layouts. Since almost everything is visible and the countdowns of all bombs and explosions are part of the game state, we use the lecture’s simplification to a fully observed Markov decision process (MDP), $o_t = s_t$:

- **Time:** one round is one episode with steps $t = 1, \dots, T$, where $T \leq 400$. In training mode the round ends when our agent dies, so death is a terminal state.
- **States s_t :** the `game_state`, with the field (walls, crates, free tiles), the bombs with their countdowns, the explosion map, the visible coins, and position, score and bomb availability of our agent (`self`) and of the opponents still alive (`others`).
- **Actions $a_t \in A = \{\text{UP, DOWN, LEFT, RIGHT, WAIT, BOMB}\}$.**
- **Rewards r_{t+1} :** the official reward is +1 per coin, +5 per kill and 0 otherwise, so it is zero in most steps. Therefore we train with a shaped reward $\hat{r}_{t+1}$ (Section 4), which can bias the policy unless the extra rewards have the form of a potential ψ , $\hat{r}_{t+1} = r_{t+1} + \gamma \psi(s_{t+1}) - \psi(s_t)$, with the discount factor γ of Section 2.2 [5, 3].
- **World probability $p^*(s_{t+1}, r_{t+1} \mid s_t, a_t)$:** the deterministic game engine together with the random order in which the actions are executed, the hidden coins and the opponents’ actions. Since the opponents’ policies are unknown, p^* is unknown too, so we use model-free RL, which learns only from observed transitions [5].
- **Policy $\pi(a_t \mid s_t)$:** the probability that our agent plays a_t in s_t ; the goal is the optimal policy π^* .
- **Training set (TS):** the observed transitions $(s_t, a_t, r_{t+1}, s_{t+1})$.

The state space is huge: 164 of the 176 free tiles can hold a crate, so there are already $2^{164} \approx 2 \cdot 10^{49}$ crate layouts, and a table with one value per state-action pair is impossible.

2.2 Value functions and Q-learning

To find the best policy of an MDP, we need value functions. We first introduce them and then show how they are learned from the TS. Usually the discounted return is used, which weights rewards

further in the future less. The agent maximizes this return, and the optimal policy maximizes its expectation [5, 6]:

$$g_t = \sum_{t'=t+1}^{T+1} \gamma^{t'-(t+1)} r_{t'}, \quad \gamma \in (0, 1], \quad \pi^* = \arg \max_{\pi} \mathbb{E}_{p^*, \pi}[g_1]. \quad (1)$$

The value function $V_{\pi}(s)$ is the expected return when we start in s and follow π , and the action-value function $Q_{\pi}(s, a)$ the expected return when we play a in s and follow π afterwards:

$$Q_{\pi}(s, a) = \mathbb{E}_{s', r \sim p^*}[r + \gamma V_{\pi}(s')], \quad V_{\pi}(s) = \mathbb{E}_{a \sim \pi(a|s)}[Q_{\pi}(s, a)]. \quad (2)$$

If p^* were known, the resulting Bellman equation could be solved by value iteration. With Q , p^* is not needed: the best action is $\arg \max_a Q(s, a)$, and the expectation over p^* is replaced by transitions from the TS. Temporal-difference (TD) learning [7] moves the current estimate towards a target return R_t :

$$Q^{(\tau)}(s_t, a_t) = Q^{(\tau-1)}(s_t, a_t) + \alpha [R_t - Q^{(\tau-1)}(s_t, a_t)], \quad R_t = r_{t+1} + \gamma V^{(\tau-1)}(s_{t+1}). \quad (3)$$

The full return $R_t = g_t$ (Monte Carlo) would need complete episodes and a very large TS; k -step targets lie in between. Since V is not learned, the lecture gives two approximations [5]:

$$V^{(\tau-1)}(s_{t+1}) \approx \begin{cases} \max_{a'} Q^{(\tau-1)}(s_{t+1}, a') & \text{Q-learning, TS: } (s_t, a_t, r_{t+1}, s_{t+1}), \\ Q^{(\tau-1)}(s_{t+1}, a_{t+1}) & \text{SARSA, TS: } (s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}). \end{cases} \quad (4)$$

Q-learning [8] uses the best next action and learns the values of the greedy policy, no matter which policy generated the data (off-policy). SARSA [9] uses the action actually played next and learns the values of the exploring training policy (on-policy), which makes it prefer safer paths. For a table of Q-values, both converge to the optimal Q-function under suitable conditions [8, 10].

We chose Q-learning for three reasons. First, the tournament plays the greedy policy $\bar{\pi}(s) = \arg \max_a Q(s, a)$, whose values Q-learning estimates. Second, being off-policy, it can learn from older transitions in a replay buffer, which deep Q-learning needs [1]. Third, SARSA's safer exploration is mostly taken over in **q_agent** by the action mask, which removes suicidal actions (Section 4). The price is the “deadly triad” of off-policy learning, bootstrapping and function approximation, under which the values can diverge [6, 11], as we also observed for **q_agent** (Sections 4.1 and 5.1.2).

2.3 Function approximation and deep Q-learning

Since a table is impossible, the lecture formulates Q-learning as a regression problem that minimizes $(R_t - Q(s_t, a_t))^2$ and generalizes from the TS to unseen states, with a linear model, a random forest or a neural network [5]. For linear regression, hand-crafted features $\phi(s)$ are defined and one weight vector β_a is learned per action, $Q(s, a) \approx \phi(s)^{\top} \beta_a$. Such a model is fast and interpretable, but only as good as its features.

A deep neural network instead learns the features itself. Deep Q-networks (DQN) [1] use a convolutional network with one output per action and two tricks: *experience replay* trains on random batches from a buffer of past transitions, so that correlated transitions of the same game are not learned together, and a *target network* $Q_{\theta-}$, an older copy of the network, computes the targets. Double DQN [2] reduces the overestimation caused by the maximum by choosing the next action with the current network and evaluating it with the target network, $R_t = r_{t+1} + \gamma Q_{\theta-}(s_{t+1}, \arg \max_{a'} Q_{\theta}(s_{t+1}, a'))$. The target network is also a common countermeasure against the divergence of the deadly triad [11]. Our two agents stand for the two sides: **q_agent** uses a linear model on 34 hand-crafted features, and **dqn_agent** a Double DQN that learns its own features from the board.

We did not consider other approaches further: policy-gradient methods such as PPO [12] learn the policy directly, but they are on-policy and typically need much more fresh game data, and tree search methods such as Monte Carlo tree search [13] are not learning methods by themselves and are limited by the 0.5s per step. Imitation learning of the `rule_based_agent` [14] would at best copy the agent we want to beat, so we concentrated on Q-learning, the main example of model-free RL in the lecture.

3 Project Planning

Author: Luis Maco

This project was developed by a team of three, Diana Krasnokutskaya, Daniela Ardila Porras, and Luis Maco, for Machine Learning Essentials in the Summer Semester 2026. The code, experiment logs, and trained models are available in the project repository: https://github.com/luisdmaco/bomberman_rl.

The work ran from registration in the week of 31.08 to the report deadline on 28.09. The main code deadline was 21.09, with an optional crash test on 17.09. This section describes the original plan, the division of responsibilities, and the changes that followed from the experimental results.

3.1 The plan

The specification prioritises systematic testing over tournament placement. We therefore recorded a before/after number for every design change, built an evaluation harness before any learning agent existed, and kept a dated experiment log (`experiments/LOG.md`) from day one so the results section could be assembled from files rather than from memory.

The specification required two models and emphasised shared work. We initially planned a component-based split covering the environment interface, learning algorithm, evaluation, and experiments, with shared feature and evaluation infrastructure. In practice, each model was carried end to end by a different team member, while evaluation, event definitions, and reward-shaping principles were shared and cross-checked. This model-based split proved easier to maintain than a strict component-based split.

We planned two structurally different models: a linear Q-learning agent over hand-engineered features as the primary tournament candidate, and a Deep Q-Network as a comparison. The DQN was allowed to fail; negative or mixed results (Section 6.2) were part of the planned comparison rather than a reason to hide the approach.

We planned a four-task curriculum with numeric gates for coin collection, safe crate bombing, hunting weaker agents, and beating `rule_based_agent`. Task 2 was the critical path because escaping bombs is crucial and most tournament losses are self-inflicted, so we placed it in week 1 to leave time for iteration.

3.2 How it actually went

Both agents reached the Task 1 gate on 06.09: the DQN collected 46.06 of 50 coins (95% CI [45.5, 46.7], zero invalid actions), and the feature-based agent matched the `coin_collector_agent` baseline. Task 2 took three reward-function fixes and roughly four days before the DQN's suicide rate on `classic` passed the 2% gate.

On 06.09. we agreed not to plot training-time measurements because the online and frozen (`train=False`) versions of the same weights can disagree substantially. This discrepancy was observed on the DQN side, but the evaluation rule applied to both agents because the tournament always runs a frozen policy. On 15.09 we built a shared comparison tool (`tools/compare_team_agents.py`) that evaluates both agents frozen, against the same seeded opponents under one protocol. Confirmed at $N=1000$ rounds on 16.09, that comparison selected the DQN (`score_hunt r1250`) as the team's tournament candidate over the committed feature-based checkpoint at that time.

The DQN arm froze on 16.09, four days ahead of schedule, after a controlled sweep (Section 6) showed that seed variation exceeded every tested hyperparameter effect. We spent the remaining days confirming the result and writing it up.

3.3 Hardware

Neither model needed a GPU. The DQN has 353k parameters, only about 6k of them convolutional, and profiling (Section 5.2) showed that gradient steps dominate the measured cost. Training and sweeps ran on one Apple M3 Max laptop with 36 GB RAM, using one CPU thread per process. Both agents also respect the tournament’s CPU-only 0.5 s-per-step budget.

4 Methods

4.1 The Q-learning agent

Author: Daniela Ardila

4.1.1 State representation

q_agent does not see the board itself, but a vector $\phi(s) \in \mathbb{R}^{34}$ of hand-crafted features (Table 1). Most of them come from a breadth-first search (BFS) from the agent’s tile over walkable tiles, i.e. free tiles without a bomb or another agent, so all distances are real walking distances. A direction is encoded one-hot as the first step of the shortest path, with an extra entry **none** if there is no target. For crates and opponents, **none** is ambiguous (no target, or already at it), and a second feature resolves it. The danger features use the blast model of the game engine: a blast reaches three tiles in each direction and is stopped only by walls, while crates are destroyed but do not block it.

Table 1: The 34 features of **q_agent**. d : BFS distance, t : steps until a blast reaches the tile.

Features	Encoding
walkable (4)	neighbour tile can be entered
danger (5)	$1/(t+1)$ for the own and the four neighbouring tiles, 0 if safe
can_bomb (1)	own bomb available
coin direction (5), distance (1)	first step to the nearest visible coin; $1/(d+1)$
crate direction (5), crates_in_blast (1)	first step to the tile that maximizes $n/(d+6)$, n = crates its blast would destroy; $n/12$ for the own tile
safe direction (5)	first step to the nearest tile outside every blast, only while in danger
opponent direction (5), distance (1), opp_in_blast (1)	first step to the nearest opponent; $1/d$ (0 if unreachable); a bomb here would reach an opponent

4.1.2 Model and learning rule

As in Section 2.3, the Q-function is linear with one weight vector per action, $Q(s, a) = \phi(s)^\top \beta_a$. All weights start at zero, and later tasks warm-start from the previous one (Section 5.1.2). After every step, the TD update (3) becomes one gradient step on $(R_t - Q(s_t, a_t))^2$:

$$\beta_{a_t} \leftarrow \beta_{a_t} + \alpha [R_t - \phi(s_t)^\top \beta_{a_t}] \phi(s_t), \quad R_t = \hat{r}_{t+1} + \gamma \max_{a' \in A(s_{t+1})} \phi(s_{t+1})^\top \beta_{a'}^-. \quad (5)$$

Besides the shaped reward $\hat{r}_{t+1}$, the target differs from the Q-learning target (4) in two ways. The maximum runs only over the allowed actions $A(s_{t+1})$ of the action mask (below), and the next value comes from a target copy β^- of the weights, which is synchronized every 100 rounds.

In addition, every weight vector is rescaled to $\|\beta_a\| \leq 10$ after each update. Both measures act against the divergence of the deadly triad (Section 2.2), which we observed as the weights of **WAIT** growing without bound (Section 5.1.2). Unlike **dqn_agent**, we use no replay buffer and no Double DQN: each transition is learned once, online, in the order it was played.

We use $\gamma = 0.95$ and a learning rate α that starts at 0.005 and decays by $\times 0.999$ per round to 0.0005. Exploration is ε -greedy over the allowed actions (a random one with probability ε [6]), with ε starting at 0.2 and decaying by $\times 0.9995$ per round to 0.05. A checkpoint is saved every 250 rounds.

4.1.3 Actions and masking

The action mask reduces A to a state-dependent set $A(s) \subseteq A$. It removes actions that are illegal, provably suicidal or useless, as well as voluntary **WAIT**, and leaves the choice among the rest to Q :

- moves onto a blocked tile or a tile that explodes in this step;
- while bombs are ticking, moves onto a tile from which no escape exists, as long as another move survives; if every move is doomed, only the move to the tile that explodes last is kept;
- **BOMB** if no bomb is available, if the blast would reach neither a crate nor an opponent, or if no escape exists after dropping it;
- **WAIT** always. It was the main cause of suicides and the action whose weights diverged, and it is only allowed again when no other action is left.

The escape check is a BFS over (tile, time) pairs that uses the lethal steps of all ticking bombs and explosions, taken from the game engine (Section 5.1.2). The mask is applied to exploration, exploitation and the target maximum, in training and in the tournament, where the agent plays the masked greedy policy $\bar{\pi}(s) = \arg \max_{a \in A(s)} Q(s, a)$. So the mask takes over most of the safe exploration that SARSA would give (Section 2.2). Since the actions are executed in random order (Section 2.1), an opponent can still enter the target tile first; the move then fails with **INVALID_ACTION**.

4.1.4 Episode boundaries

Every end of a round is treated as terminal, $R_t = \hat{r}_{t+1}$: death, which ends the round in training mode, and the 400-step limit. Strictly, since the step counter is not a feature, the step limit should be treated as a truncation and bootstrapped [4], as **dqn_agent** does. We kept the terminal target, which affects only the last transition of rounds that reach step 400.

4.1.5 Reward shaping

The shaped reward $\hat{r}_{t+1}$ of the final model is the sum of the event rewards in Table 2 and a step penalty of -0.03 . Besides the opponent events of Section 5.1.2, **IN_DANGER** fires on a tile inside a current or coming blast and **ESCAPED_DANGER** when leaving one. **BOMBED_CRATES** pays per crate at the moment the bomb is dropped, not four steps later, and **STALLED** fires when the last 10 positions lie on at most two tiles. **AMBUSH_READY** marks a bomb that is ready while an opponent stands in its blast and an escape exists, and **WIN_ROUND** a round that ends with more points than every opponent still alive. Since the environment emits **GOT_KILLED** together with **KILLED_SELF**, the former is dropped on a suicide, so a suicide costs -15 and not -23 .

Only the distance terms resemble potential-based shaping (Section 2.1): with $\psi(s) = -cd(s)$ and $c = 0.1$ or 0.15 , one step closer earns $\psi(s_{t+1}) - \psi(s_t) = c$. Even these are not exact, since they leave out γ and pay $\pm c$ however far the distance jumps, e.g. when a coin is collected or a crate opens a path. All other terms are not potential-based, and **STALLED** even depends on past positions. So $\hat{r}$ can change the optimal policy [3]. We accept this bias and judge every change only by the official score in frozen evaluation.

Table 2: Event rewards of the final `q_agent`; engine events above the line, custom events below.

Event	$\hat{r}$	Event	$\hat{r}$
COIN_COLLECTED	+1	KILLED_OPPONENT	+6
KILLED_SELF	-15	GOT_KILLED	-8
CRATE_DESTROYED	+0.5	COIN_FOUND	+0.3
SURVIVED_ROUND	+0.5	INVALID_ACTION	-1
BOMBED_CRATES (per crate)	+0.3	STALLED	-0.3
IN_DANGER	-0.15	ESCAPED_DANGER	+0.5
closer / farther to coin	± 0.1	closer / farther to opponent	± 0.15
BOMBED_OPPONENT	+0.8	TRAPPED_OPPONENT	+2.0
AMBUSH_READY	+0.3	WIN_ROUND	+1.5
WAITED_WITH_OPPONENTS	-0.1	every step	-0.03

4.1.6 Evaluation methodology

All `q_agent` results use frozen policies (`train=False`): no exploration and no updates, so a fixed seed gives exactly the same games. `evaluate_models.py` loads every checkpoint and plays 100 rounds on `classic` with seed 42. It reports score, kills, suicides, coins and crates per round, and the margin, our score minus the best opponent’s score. Checkpoints are ranked by margin, and we keep the best one, which is usually from the middle of a run. Since the saved statistics contain only totals per agent, win rates come from 100 separate one-round games with seeds 0–99. These evaluations use fewer rounds than those of `dqn_agent` and no confidence intervals. The comparison of the two agents uses the shared protocol of Section 6 (margin against the mean opponent, 95% confidence intervals).

4.2 The DQN agent

Author: Luis Maco

4.2.1 State representation

`dqn_agent` reads six board channels: walls, crates, self, opponents, coins, and danger countdowns. The channels remain fixed across tasks, including the channels that are empty on Task 1. The board is represented either globally at 17×17 or as a padded 13×13 window centred on the agent (`ego:13`); this choice is stored with the checkpoint because the tournament provides no environment variables.

4.2.2 Network and learning rule

The agent is a Double DQN [2] with a target network synchronised every 1000 gradient steps and a uniform replay buffer of 50,000 transitions. Double DQN separates action selection from evaluation in the bootstrap target, reducing the overestimation bias of vanilla DQN [2]. A small convolutional front end feeds a fully connected head with one output per action.

4.2.3 Actions and masking

All six actions are available at the architecture level. A legal-action mask removes moves into blocked or immediately lethal tiles and is also applied to the Double-DQN target. An optional suicide mask removes BOMB when breadth-first search finds no safe escape within the four-step fuse; from Task 3 onward it also considers tiles an opponent can reach first. The suicide mask is enabled only at play time, while the legal-action mask is always active.

4.2.4 Episode boundaries

A round that reaches the 400-step limit is treated as a truncation, following Pardo et al. [4]. It is not assigned a terminal target with value zero, which previously combined with potential shaping to reward being far from a coin at timeout.

4.2.5 Reward shaping

The base reward is sparse, so `dqn_agent` adds potential-based shaping and custom events. Following Ng, Harada and Russell [3], it uses $F(s, s') = \gamma_{\Phi} \Phi(s') - \Phi(s)$, with $\Phi(s)$ the negative breadth-first-search distance to the nearest reachable coin (or, once no coin is visible, to the nearest tile from which a bomb would break a crate). Under the standard potential-based shaping conditions, this form leaves the optimal policy unchanged when γ_{Φ} equals the agent’s discount factor γ . We deliberately break that equality (Section 5.2) and cap the potential, so the exact invariance guarantee no longer applies; these changes removed a reward exploit observed in practice.

The event rewards distinguish useful, useless, and suicidal bombs, while danger events shape escape. They use information already available in the game state and event list, so they fit within the 0.5s step budget.

The training profiles are `curriculum` for Tasks 1–2, `score` for scoreboard events plus safety shaping, and `score_hunt`, which adds a bonus for an escapable bomb threatening an opponent. We introduced them after observing a mismatch between training reward and the tournament scoreboard.

4.2.6 Symmetry augmentation

The `classic` board is invariant under eight square symmetries. Augmentation is applied when sampling from replay, per sample rather than per batch, so the original transition is stored once and transformed orientations are sampled independently. It affects training only. Action permutations were checked by comparing transformed states with recomputed features for all eight group elements.

4.2.7 Evaluation methodology

All DQN results use frozen policies (`train=False`). Comparisons use fixed seeds, $N \geq 200$ rounds, and $N = 1000$ near gate thresholds; a gate passes only when its whole 95% confidence interval clears the threshold. Because `round_statistics` aggregates all agents, per-agent statistics are used when opponents are present.

5 Training

5.1 The Q-learning agent

5.1.1 Tasks 1 and 2: from coin collector to crate bomber

Author: Daniela Ardila

Task 1. The first version played `coin-heaven` with the walkable, danger, bomb and coin features of Table 1. Before it could learn anything, I had to fix three bugs: the feature vector was assembled wrongly, the weights were saved to the wrong folder (the callbacks run inside the agent’s directory), and the `BOMB` mask was inverted. The agent then got stuck moving back and forth between two tiles. Symmetric coin-approach rewards (± 0.1) and a decaying ε fixed this, and the agent matched the `coin_collector_agent`: the same number of coins per round, with 0.61 more steps.

Task 2. For crates I added the crate direction, `crates_in_blast` and the safe direction, which brought the vector to 27 entries. I first trained on `loot-crate`, but switched to `classic`, because `loot-crate` has 50 coins instead of 9 and its weights did not transfer well. Four problems appeared one after another, each one visible in the frozen evaluation:

1. *The agent stopped bombing.* `IN_DANGER` (-0.3 per step) cost more than the crates a bomb destroyed ($+0.5$ each), so even a successful bomb was net negative. I lowered it to -0.05 (-0.15 in the final model, Table 2) and added `BOMBED_CRATES`, which pays per crate already when the bomb is dropped.
2. *It killed itself in 97% of rounds.* With $\gamma = 0.9$ and `KILLED_SELF` = -5 , the discounted death a few steps after a bomb was cheap compared with the about $+26$ shaped reward of a whole round. I raised γ to 0.95 and the penalty to -15 , added `ESCAPED_DANGER` and masked `WAIT` inside blast zones. Suicides dropped to zero.
3. *Training diverged after about 2500 rounds*, with the weights of `WAIT` growing about tenfold under a constant learning rate. A decaying learning rate and a checkpoint every 250 rounds were enough here. The same problem came back in task 3 (Section 5.1.2).
4. *The bombs were badly placed.* The agent dropped as many bombs as the `rule_based_agent`, but destroyed 1.31 crates per bomb instead of 3.07. Allowing only spots with at least three crates backfired: 7% more crates per bomb, but 37% fewer bombs because of detours. The rate-based target $n/(d+6)$ (Table 1) raised the yield to 2.40 crates per bomb and the number of bombs to 45.9 per round.

The best checkpoint (round 4500) collected 7.89 of 9 coins per round on solo `classic` (150 rounds, seed 42) without suicides, about 92% of the `rule_based_agent`’s 8.58. After these fixes, all checkpoints from round 2000 to 5500 scored between 7.09 and 7.89, so training was stable. This agent was the starting point of task 3.

5.1.2 Task 3: from crate-bomber to hunter

Author: Diana Krasnokutskaya

At the start of task 3 the Q agent was a strong solo player (7.89 of 9 coins per round on `classic`, no suicides), but it had never seen an opponent: none of its 27 features described other agents, and its bomb logic assumed that only its own bomb exists. Before adding anything new, we checked the existing danger model against the environment source, because a hunting agent that misjudges blasts will kill itself long before it kills anyone.

Correcting the blast model. Our `get_blast_coords` stopped a blast *after the first crate*, whereas the environment (`items.Bomb.get_blast_coords`) stops it only at walls: crates are destroyed and the tiles behind them explode too. The old version therefore declared tiles behind a crate safe although they were lethal. Solo this rarely mattered, but with opponent bombs it becomes a direct cause of death. we fixed the function and renormalised `crates_in_blast` by 12 (the true maximum, $3 \text{ tiles} \times 4 \text{ directions}$) instead of 4.

A time-aware escape check. The old `escape_exists` asked a single question: can we reach a tile outside *my own hypothetical* bomb’s blast within four steps? It ignored bombs that were already ticking, including opponents’ bombs, as well as explosions that were still burning. I rewrote it as a breadth-first search over (tile, time) pairs. we took the timing directly from `environment.do_step`: a bomb with timer t is lethal at the end of steps t and $t + 1$, because the explosion persists one step, and the current explosion map is lethal at step 0. The search moves a set of reachable safe tiles forward one step at a time. Waiting counts as a move, stepping back onto the tile of the bomb just dropped does not, and the escape succeeds if at least one tile survives until the last lethal step. The same function has two uses:

- in the **action mask**, with `assume_own_bomb=True`, to forbid BOMB when no escape exists, and with `assume_own_bomb=False`, to prune moves onto tiles from which there is no way out. Moves are pruned only while at least one other move survives, so the mask never hands the agent to the WAIT fallback because of an over-strict check;
- in the **reward function**, from the opponent’s point of view, to detect bombs that trap an opponent (see below).

We also tightened the BOMB mask. Before, bombs were forbidden only when no crates were left on the board. Now BOMB is allowed only when the blast from the current tile actually reaches a crate or an opponent.

Opponent features. We added three feature groups, which brings the vector from 27 to 34 entries (Table 3).

Table 3: Opponent features added for task 3.

<code>opp_dir_up,down,left,right,none</code>	one-hot, first step of a BFS to the nearest reachable opponent	where to go to hunt
<code>opp_dist</code>	$1/d$, 0 if no opponent is reachable	how urgent / how close
<code>opp_in_blast</code>	1 if a bomb dropped here would reach an opponent	when to drop

The BFS treats occupied tiles as blocked but returns as soon as a neighbouring tile holds an opponent, so the distance is the real walking distance to “adjacent to the opponent”. `opp_dir = none` is ambiguous on its own (no opponent reachable, or already next to one), and `opp_in_blast` and `opp_dist` resolve that ambiguity. This follows the same pattern the task-2 features used for crates.

Reward shaping for hunting. A linear model can only learn to hunt if the reward tells it that approaching and bombing an opponent is good long before the kill (+5) happens. We added four custom events (Table 4).

Table 4: Opponent-directed custom events and their initial rewards.

Event	Fires when	Initial reward
BOMBED_OPPONENT	a just-dropped bomb’s blast covers an opponent	+0.5
TRAPPED_OPPONENT	additionally, that opponent has no escape (checked with <code>escape_exists</code> from its position, including the new bomb)	0.0
MOVED_CLOSER_TO_OPPONENT	BFS distance to the nearest opponent decreased	+0.05
MOVED_FARTHER_FROM_OPPONENT	BFS distance increased	−0.05

The approach terms are symmetric so that the agent cannot collect reward by walking back and forth. We kept them weaker than the coin-approach terms (± 0.1) on purpose, so that hunting never outbids collecting coins and staying safe. TRAPPED_OPPONENT was implemented and logged but started with a reward of 0: We first wanted to see how often it fires before pricing it. During task 3, raising BOMBED_OPPONENT and TRAPPED_OPPONENT together to 1.0 and 2.0 was tried twice and lost both times in evaluation, so we left them at 0.5 and 0.0 in my commit.

We also found an unintended double penalty. The environment emits GOT_KILLED together with KILLED_SELF on every suicide. We introduced a separate GOT_KILLED penalty of −8 for being killed by an opponent, and `reward_from_events` now drops GOT_KILLED whenever KILLED_SELF is present. Without this filter a suicide would have cost −23 instead of the intended −15.

Training logs. Until then, training was judged only by evaluating checkpoints afterwards. We added a per-round CSV log (`training_log.csv`) with steps, total shaped reward, mean absolute TD error *before* each update, ε , learning rate, and counts of coins, crates, bombs, kills, suicides, deaths, invalid actions and the two opponent-bomb events. It paid off right away. When Daniela continued the task-3 training, the log showed kills and reward getting worse over a 5000-round run while WAIT’s weight vector grew without bound. That was the evidence for her diagnosis of maximization-bias divergence and for the fixes in Section 4.1: weight-norm clipping, a target network, and banning voluntary WAIT. It also showed that TRAPPED_OPPONENT fired regularly while carrying no reward, which led to setting it to +2.0, the most effective single change of task 3.

Training setup. Training ran online, with one TD update per step bootstrapping from the masked maximum over the next state’s allowed actions, on `classic` against `peaceful_agent` and `coin_collector_agent`. The agent warm-started from the task-2 weights plus seven new feature weights and kept the task-2 hyperparameters ($\varepsilon = 0.2$ decaying $\times 0.9995$ per round to 0.05, decaying learning rate, $\gamma = 0.95$, step penalty -0.02). A checkpoint was saved every 250 rounds, because the best model is often found in the middle of a run, and each checkpoint was evaluated frozen with `evaluate_models.py` (100 rounds, seed 42) and ranked by margin.

5.1.3 Task 4: from hunter to rule_based_agent

Author: Daniela Ardila

Last changes in task 3. Diana’s training log showed the weights of WAIT growing without bound (Section 5.1.2). The state after WAIT has almost the same features as before, so the target of WAIT contained its own current value, and every overestimate was fed straight back into it. I updated Diana’s code with four fixes against this divergence (Section 4.1): the weight vectors are clipped to norm 10, the learning rate starts at 0.005 and decays, the target uses a copy β^- of the weights that is synchronized every 100 rounds, and voluntary WAIT is banned from the action mask, with a small penalty for the remaining WAITs while opponents are on the board (WAITED_WITH_OPPONENTS). I then added two events that point the agent towards winning instead of collecting shaped reward: AMBUSH_READY pays for standing with a ready bomb whose blast would reach an opponent while an escape exists, and WIN_ROUND pays at the end of a training round with more points than every opponent still alive. I also updated Diana’s reward function to the final values of Table 2. Compared with Section 5.1.2, TRAPPED_OPPONENT rose from 0 to +2.0, BOMBED_OPPONENT from 0.5 to 0.8, the opponent-approach terms from ± 0.05 to ± 0.15 and the step penalty from -0.02 to -0.03 . With these changes, the agent reached a margin of +5.14 against the weak opponents (Table 6).

Starting point. I continued from this checkpoint. Against three `rule_based_agents`, however, it scored only 4.16 against 3.18 for the best opponent, with 0.55 suicides per round. So the missing skill was surviving a crowded board.

Training setup. Only the opponents changed: three `rule_based_agents` on `classic` for 6000 rounds. Features, rewards, mask and hyperparameters stayed the same. The agent was warm-started from the task-3 weights, loaded before the target copy β^- is made, so both start from the same weights. I restarted ε and α at 0.2 and 0.005, because the new opponents make part of the old estimates wrong, so the agent should explore again.

Training log. Between the first and the last 250 rounds, the shaped reward rose from 29.6 to 45.5 and the kills from 0.22 to 0.39 per round, while the suicides fell from 0.54 to 0.39. There were no signs of divergence this time.

Checkpoint selection. I evaluated all 24 checkpoints and the starting model frozen against one `rule_based_agent` (100 rounds) and ranked them by margin. The best was round 2500 with +3.94, against +2.55 for the starting model. The last checkpoint reached only +1.27, so, as in task 3, the best model came from the middle of the run. The seed fixes only the world, not the `rule_based_agent`’s random choices: the same checkpoint measured twice differed by about 0.1 points, so small differences between checkpoints are not meaningful at $N = 100$. Against three `rule_based_agents`, round 2500 scored 5.00 against 2.77 for the best opponent, with 0.22 suicides per round. It won 50% of 100 single rounds outright and tied 11%, where an agent as strong as its opponents would win about 25%. This checkpoint is the final `q_agent`. Section 6.3 confirms it at $N = 1000$ and compares it with `dqn_agent`.

5.2 The DQN agent

Author: Luis Maco

5.2.1 Curriculum

Training followed the four-task curriculum, resuming weights between stages. On Task 1 (`coin-heaven`), `BOMB` and `WAIT` were removed from exploration because an $\varepsilon=1$ policy could otherwise die before collecting experience. Task 1 does not require bombs, so the mask was lifted for Task 2. We also used a temporary `crate-light` scenario with crate density 0.35 instead of the tournament’s 0.75; it exists only in `settings.py`.

5.2.2 Opponent choice as a training variable

From Task 3 onward, opponent choice had a substantial effect. Two 4000-round runs resumed from the same checkpoint and differed only in the opponent: the agent scored 81 kills, flat over time, against `coin_collector_agent`, but 258 against `rule_based_agent`, rising from 0.010 to 0.147 kills per round. The former ignores the agent; the latter approaches and bombs it. We therefore used three copies of `rule_based_agent` as the Task 4 training partners rather than the easier opponents used earlier in the curriculum.

5.2.3 Training efficiency

Profiling one training round showed gradient steps at 92% of the cost, compared with 5% for acting and 2% for feature extraction. We therefore stored replay data as `uint8` instead of `float32`, changed updates from 32 samples every 4 steps to 128 every 16, and ran ablations as separate single-threaded processes. Six configurations covering roughly 7.2 million environment steps then completed in 51 minutes instead of an estimated five hours sequentially.

5.2.4 Reward shaping problems found and fixed during training

Two reward problems surfaced during training. The potential-based shaping term, computed with the same discount as the agent’s $\gamma = 0.95$, leaves a residual of $\Phi(s)(\gamma - 1)$ on every step that makes no progress; at a coin ten steps away this residual paid roughly +0.065 every two steps for oscillating in place, for as long as the agent chose to oscillate, and once the next coin was twelve or more steps away, collecting the current one was net negative under the shaping term. The fix decouples the shaping discount from γ (set to 1.0, which removes the per-step drift so the step penalty makes hovering strictly negative) and caps the distance term, bounding the size of the jump when a coin is collected. Separately, once crates entered the curriculum, the potential function’s target (the nearest reachable coin) was undefined on every Task 2 board, because every coin starts hidden inside a crate. The Task 1 shaping signal was therefore absent on Task 2 until the potential was retargeted to the nearest good bombing spot when no coin is visible. Both fixes

were checked against the arithmetic before another training run was started: a five-step expected-value comparison of “bomb two crates” against “do nothing” moved from -0.44 to $+2.67$ once the four Task 2 reward fixes (including suppressing the “entered danger” penalty on the step a bomb is deliberately dropped) were applied together.

5.2.5 Checkpointing and instability

Checkpoints were evaluated frozen, and the best rather than the last checkpoint was kept. Most from-scratch egocentric-view runs peaked while ε was still around 0.84, then decayed as invalid actions rose from 0 to 40–80 per round. This is consistent with online-DQN instability. The global-view run improved monotonically, leaving the egocentric crop as a possible contributor and an open problem (Section 6.2).

5.2.6 Reward-profile progression toward the tournament objective

The Task 2 curriculum profile paid for crate destruction that had no direct scoreboard equivalent: `CRATE_DESTROYED` contributed roughly $+24$ training reward per round. The `score` profile was safer but did not meet the tournament objective. We therefore added `score_hunt`, a bonus for an escapable bomb threatening an opponent. Hand-built boards verified that it did not fire for suicidal bombs or crate-only demolition, and the profile improved the tournament margin (Section 6.2).

5.2.7 Fine-tuning

When a policy overfit to one training distribution and lost general ability (Section 6.2), a short, low-learning-rate fine-tune from the same checkpoint was sufficient: 1500 solo rounds at one fifth of the normal learning rate, 7.6 minutes of wall clock, recovered solo coin collection from 4.95 to 7.87 coins while keeping the tournament gate the model had already passed.

6 Experiments and Results

6.1 The Q-learning agent

6.1.1 Tasks 1 and 2

Author: Daniela Ardila

All numbers in this section are frozen evaluations: task 1 on `coin-heaven` (100 rounds), task 2 on `classic` without opponents (150 rounds, seed 42). For every task I kept the best checkpoint, but it could not be used as the start of the next task: each task added features (16 in task 1, 27 in task 2, 34 from task 3 on), so the saved weights no longer matched the length of the feature vector, and I trained each run from zero. Only the step from task 3 to task 4 reused the best checkpoint, because no features were added (Section 6.1.3).

Task 1. The agent collected all 50 coins in every round, the same as the `coin_collector_agent`, and needed 124.5 steps per round against 123.9. So on an empty board, the learned policy is as good as the BFS baseline.

Task 2. Figure 1 shows every checkpoint of the three task-2 runs, which were trained after successive changes to rewards, mask and features (Section 5.1.1). Only the last run came close to the `rule_based_agent`. Even in this run the coins jump between 2.0 and 7.9 from one checkpoint to the next, so the last checkpoint is not a good choice, and the evaluation of all checkpoints decided which model to keep. Table 5 compares the best checkpoint (round 4500) with the `rule_based_agent`. It reaches 92% of its coins and never kills itself. The remaining gap comes

from bomb placement: the agent destroys fewer crates per bomb (2.40 against 3.07) and makes up most of it by dropping more bombs.

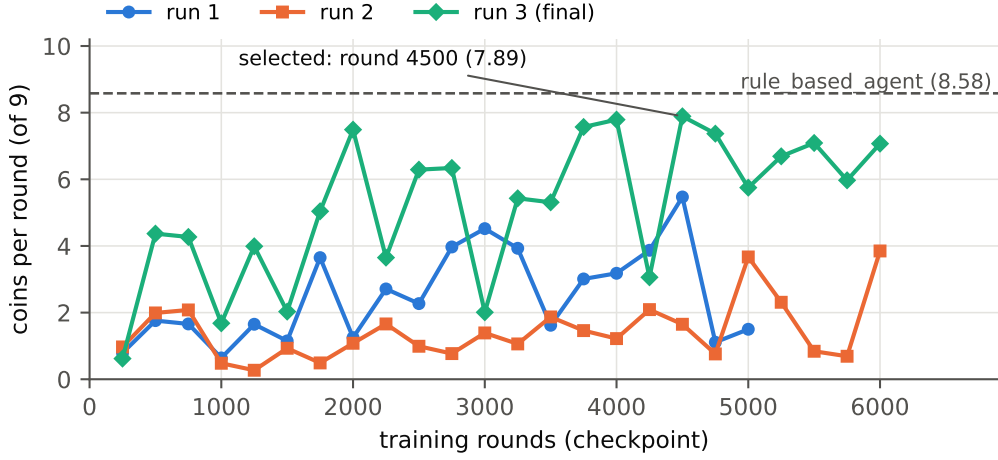


Figure 1: Coins per round of every 250-round checkpoint of the three task-2 runs (frozen, solo `classic`, 150 rounds, seed 42). The dashed line is the `rule_based_agent`.

Table 5: Best task-2 checkpoint against the `rule_based_agent` (solo `classic`, 150 rounds, seed 42, values per round).

Agent	Coins	Crates	Suicides	Bombs	Crates/bomb
q_agent (round 4500)	7.89	109.9	0.00	45.9	2.40
rule_based_agent	8.58	116.6	0.00	38.1	3.07

6.1.2 Task 3

Author: Diana Krasnokutskaya

All numbers in this section are frozen evaluations on `classic` against `peaceful_agent` + `coin_collector_agent`, 100 rounds, seed 42. Margin is our score minus the best opponent’s score (Table 6).

Table 6: Q agent on task 3 (frozen, 100 rounds, seed 42).

Model	Score	Kills	Suicides	Coins	Crates	Margin
task-3 checkpoint (<code>model_3_diana</code>), first evaluation	5.78	0.41	0.22	3.73	51.5	−1.30
best checkpoint of the next run (r2500)	7.31	0.56	0.27	4.51	58.1	+0.68
after fixes + <code>TRAPPED_OPPONENT</code> = +2 (r1000)	9.78	0.83	0.29	5.63	63.6	+5.14
<code>model_3_diana</code> , re-evaluated with the final mask	6.41	0.39	0.25	4.46	54.6	−0.40

My checkpoint learned the new behaviour qualitatively: it achieves 0.41 kills per round on average, and it destroys about as many crates as the task-2 agent. It did not yet win, though. With a margin of −1.30 it still scored less than the best opponent, usually the coin collector, which competes for the same coins and is not slowed down by bombing crates. The re-evaluation in the last row loads the same weights with the later version of the action mask (voluntary `WAIT` banned, a different fallback). The margin improves by 0.9 without any retraining. This shows how much of the agent’s behaviour comes from the mask rather than from the learned weights, and it is a caveat for any comparison of Q-agent checkpoints evaluated under different code versions.

The large jump to +5.14 came from the next step, which builds on this work: Daniela’s fixes for the divergence found in my training log, together with pricing `TRAPPED_OPPONENT` at +2.0. Over the 3000 training rounds behind that checkpoint, 82.5% of rounds ended with our agent ahead of both opponents. The general lesson of task 3 is that for a linear model **the event that predicts the outcome must carry reward**. `BOMBED_OPPONENT` fires for every bomb near an opponent, most of which the opponent simply walks out of. `TRAPPED_OPPONENT` fires only for bombs the opponent cannot escape, and those are almost always kills. As long as it carried zero reward, the agent had no reason to prefer the bombs that actually work.

To connect task 3 to the tournament, my checkpoint was later also evaluated against three `rule_based_agents` ($N = 200$, seed 157): score 3.065, margin +0.233 against the mean opponent, strictly first in 23.5% of rounds, but **suicide in 61.5% of rounds**. Hunting weak opponents and surviving a crowded board are different skills. The later task-4 training of the Q agent (Section 5.1.3) improved exactly this weakness.

6.1.3 Task 4 and the final model

Author: Daniela Ardila

All numbers except Figure 2 are frozen evaluations on `classic`, 100 rounds, seed 42, against one or three `rule_based_agents`. The seed fixes only the world, not the random choices of the `rule_based_agent`: the same checkpoint measured twice differed by about 0.1 points, so smaller differences are not meaningful.

Reusing the task-3 checkpoint. Since task 4 used the same 34 features, the best task-3 checkpoint was the starting point. Figure 2, the only training-time measurement here (online, with exploration), shows that training improved quickly from this start: within the first 1000 rounds, the suicides fell from 0.54 to about 0.38 per round and the kills rose from 0.22 to about 0.35. After that, both stayed flat for the remaining 5000 rounds, so longer training with the same setup was unlikely to help.

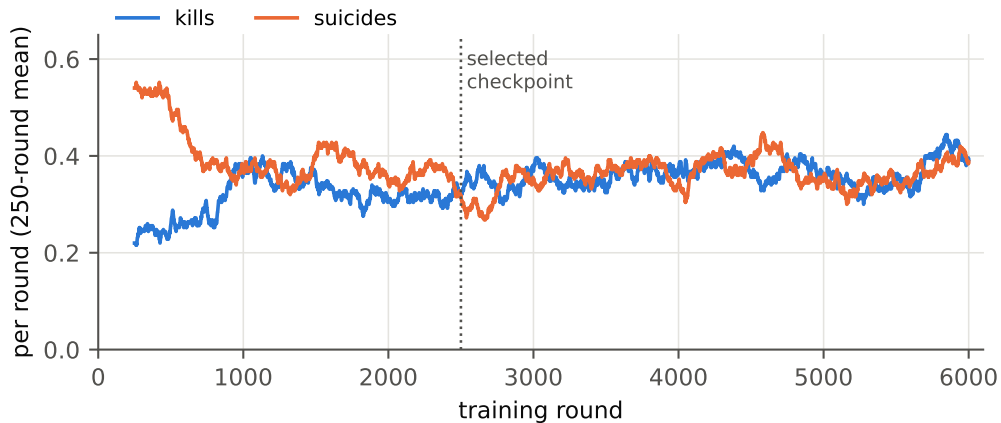


Figure 2: Kills and suicides per round during task-4 training against three `rule_based_agents` (online, with exploration; mean over 250 rounds, from `training_log.csv`).

Selecting the final model. Figure 3 shows the margin of every checkpoint against one `rule_based_agent`. As in task 2, the checkpoints vary a lot. The best one is in the middle of the run: round 2500 reached +3.94, against +2.55 for the starting model, while the last checkpoint reached only +1.27. So round 2500 became the final `q_agent`.

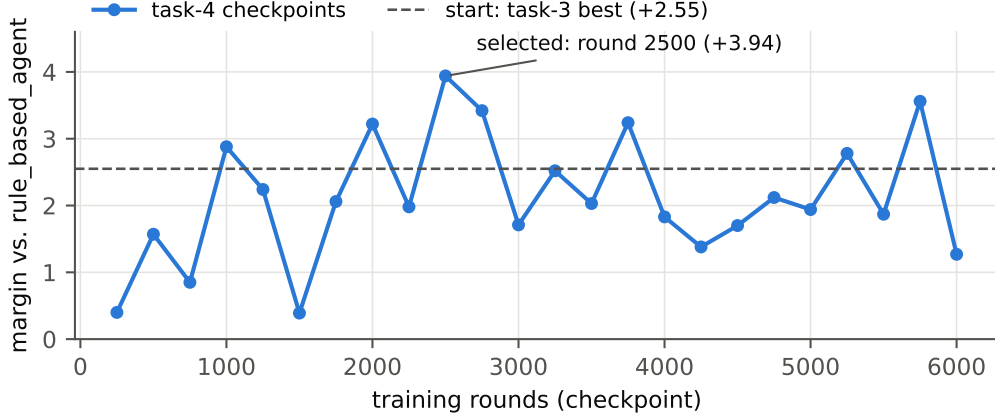


Figure 3: Margin (own score minus opponent’s score) of every 250-round task-4 checkpoint against one `rule_based_agent` (frozen, 100 rounds, seed 42).

Table 7 compares the starting model and the final model. Against one opponent, the final model wins 86% of the rounds. Against three, it scores almost twice as much as the best opponent and wins 50% of the rounds, where an agent as strong as its opponents would win about 25%. The largest change is safety: with three opponents, the suicides fell from 0.55 to 0.22 per round, which was exactly the weakness of the task-3 agent (Section 6.1.2). With 1000 rounds on a different seed, Section 6.3 confirms this result (score 5.13, suicides in 25.5% of the rounds).

Table 7: Starting model (best task-3 checkpoint) and final model (task-4 round 2500) against `rule_based_agents` (frozen, `classic`, values per round). Win rates come from 100 single-round games with seeds 0–99.

Opponents	Model	Score	Best opp.	Kills	Suicides	Win / tie / loss
1×	start	6.34	3.79	0.22	0.35	–
	final	7.03	3.09	0.22	0.19	86 / 0 / 14 %
3×	start	4.16	3.18	0.29	0.55	–
	final	5.00	2.77	0.36	0.22	50 / 11 / 39 %

6.2 The DQN agent

Author: Luis Maco

All numbers below are measured frozen (`train=False`), with $N \geq 200$ rounds and a fixed seed list unless noted, and a gate counts as passed only when the whole 95% confidence interval clears its threshold.

6.2.1 Baselines

On `coin-heaven`, an untrained network acting under an ϵ -greedy random policy averages about 18 coins with roughly 145 invalid actions per 400-step round; `rule_based_agent` averages 50.0 coins in 125.2 steps with zero invalid actions. On solo `classic`, `rule_based_agent` averages 8.60 coins with 0% suicide; in the four-player tournament shape (three copies of `rule_based_agent`) it averages a 3.30 mean score with 6.9 invalid actions per round of its own.

Task	Gate	Result	Value	N
1, <code>coin-heaven</code>	$\geq 45/50$ coins, 0 invalid	PASS	46.06 coins, CI [45.5, 46.7]	200
2, <code>classic</code> survival	suicide $< 2\%$	PASS	0.00%, 95% upper 0.38%	1000
2, <code>classic</code> coins	$\geq 8/9$ coins	FAIL	2.37 ± 0.12	1000
3, vs <code>peaceful_agent</code>	kill in $\geq 80\%$ of rounds	FAIL	3.0% [95% 2.1, 4.3]	1000
3, vs <code>coin_collector_agent</code>	positive margin	PASS	$+0.89 \pm 0.35$	400
4, vs <code>3xrule_based_agent</code>	mean score strictly above	PASS	4.961 ± 0.202 , margin $+2.250 \pm 0.238$	1000

Table 8: Final candidate (`score_hunt` r1250) against every curriculum gate. Task 4 is the specification’s requirement; Tasks 1–3 are our own curriculum checkpoints.

6.2.2 Gate results

Task 1 and Task 4 pass their gates. The selected checkpoint is weaker on the Task 2 solo test and against the passive opponent in Task 3. An earlier checkpoint (`task4-C-solo-refresh-r1000`, called *C* below) reached 7.70 ± 0.11 solo coins and 76.7% kill-in-round against `peaceful_agent`, closer to those gates, but its tournament margin against the weakest of three `rule_based_agents` was only $+0.54 \pm 0.28$, against $+2.18 \pm 0.27$ for the selected checkpoint. Both pass Task 4, while checkpoint C performs better on the solo and passive-opponent tests.

6.2.3 Measurement and implementation bugs

Four bugs were identified through measurement.

1. **Exploration suicide.** With BOMB in the random exploration set, the agent killed itself within a few steps on `coin-heaven`, ending training before learning began. We masked BOMB for Task 1.
2. **Online policies are not frozen policies.** The same saved weights scored 45 coins online while the network was updating and 6.2 frozen, a factor of seven. All subsequent evaluations therefore use frozen policies.
3. **Reward shaping paid for idleness.** Covered in Section 5.2: the discounted potential term rewarded oscillating near a coin and made collecting a distant coin net negative in the worst case.
4. **Cached next_state equalled state.** A transition cache keyed on (`round`, `step`) collided, because the environment increments its step counter once, before an agent acts, so the state passed to `act()` and its successor carried the same key. Instrumentation showed that 800 of 800 sampled transitions had `next_state == state` before the fix, and 35.2% after, consistent with moves that leave the agent in the same position. With $s' = s$, the Bellman target becomes $r + \gamma \max_a Q(s, a)$ for the same state, preventing value from propagating correctly. Wall avoidance was learned almost perfectly, while multi-step coin collection was not.

Before the fourth bug was fixed, a six-configuration ablation varying shaping, view, learning rate, and batch settings placed all configurations between 2.95 and 5.20 frozen coins with overlapping intervals. This lack of separation led to the transition-cache bug.

6.2.4 Specialization under training against `rule_based_agent`

A checkpoint trained purely against three `rule_based_agents` (`task4-classic-r3250`) passed Task 4 with a $+0.78 \pm 0.25$ margin at $N=1000$. Against the other boards, it lost ground relative to its Task 2 parent: solo `classic` coins fell from 7.51 to 4.34, and kill-in-round against `peaceful_agent` fell from 69.2% to 37.7%, while its self-kill rate did not get worse (0.60% $\rightarrow$

0.50% solo; 42.5% $\rightarrow$ 33.2% in the crowded game). The bombing rate also fell, from 47.6 bombs per solo round for the parent to 26.5 for this checkpoint. The four thousand rounds against **rule_based_agents** are consistent with caution that reduces crate destruction and access to hidden coins in solo play. This suggests opponent-distribution overfitting on the same seven boards. The final **score_hunt** checkpoint shows the same pattern. Its bombing rate depends on the opponent: it is 44.2 bombs per round against three **rule_based_agents**, 32.5 against **coin_collector_agent**, and only 12.5 both solo and against **peaceful_agent**. This is consistent with its stronger performance on the required gate and weaker performance on the solo-coin and peaceful-hunting curriculum tests that earlier checkpoints had passed. Task 4 is the specification’s requirement; Tasks 1–3 are the project’s curriculum checkpoints.

6.2.5 Hyperparameter sweep and seed sensitivity

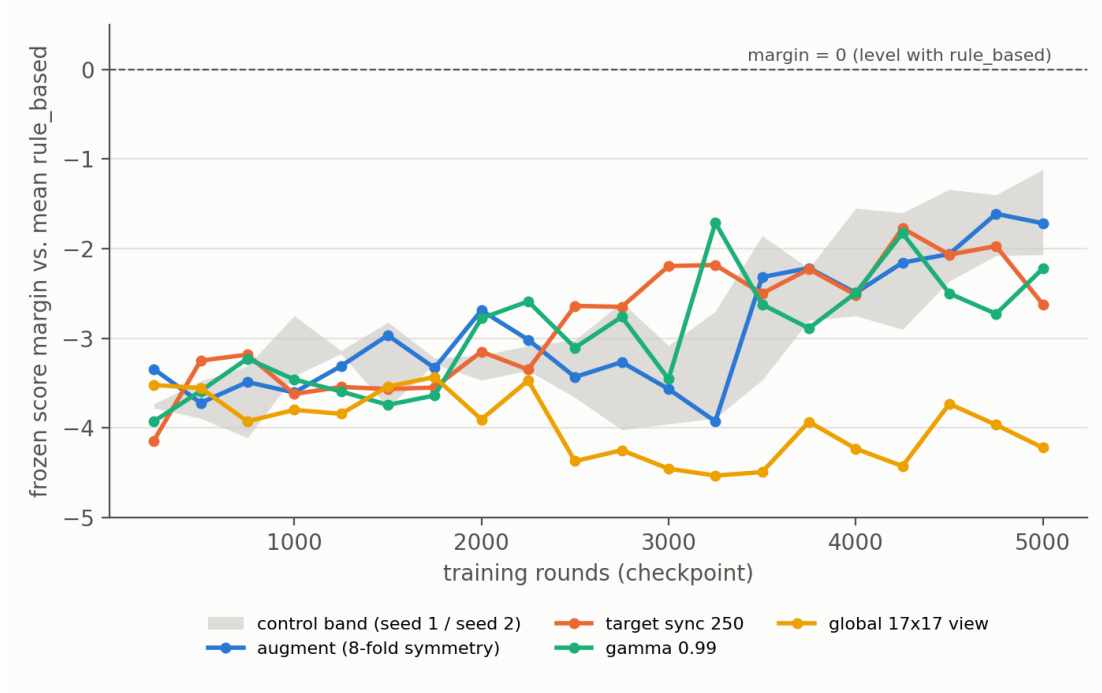


Figure 4: Frozen margin against the mean **rule_based_agent** for every 250-round checkpoint of the task-4 sweep, 60 evaluation rounds each. The grey band spans the two control runs, which differ only in their random seed. Values below zero mean the agent scores less than the rule-based agents.

Six from-scratch configurations were trained for 5000 rounds on **classic** against three **rule_based_agents** and ranked over the last five checkpoints (300 pooled evaluation rounds each). Three changed one hyperparameter relative to control; a fourth used the global view, and the control was repeated with another seed.

Configuration	Pooled margin	Suicide	Kill/round
control (seed A)	−1.401	31.0%	12.0%
augmentation	−2.008	46.7%	12.0%
target sync every 250	−2.190	46.7%	12.0%
discount 0.99	−2.354	53.0%	9.7%
control (seed B)	−2.431	54.0%	10.7%
global view	−4.117	6.0%	1.0%

Table 9: Task-4 hyperparameter sweep, ranked on pooled margin over the last five checkpoints of a 5000-round from-scratch run.

The two controls differ by 1.030 margin points. The augmentation, target-sync, and discount configurations all fall between the two observed control results, at margins from -2.354 to -2.008 . The global-view configuration falls outside this range at -4.117 , 1.686 points below the worse control. This differs from the Task 1 result, where the global view was the only configuration that kept improving instead of decaying. Its low 6.0% suicide rate does not by itself indicate learned bomb safety: the same configuration scored 0.15 and killed an opponent in only 1% of rounds. No from-scratch run closed the roughly two-point gap to the incumbent within budget. The runs had not clearly flattened by round 5000; the control’s margin moved $-3.17 \rightarrow -3.02 \rightarrow -2.22 \rightarrow -1.12$ across the run. The results therefore do not establish that the tested factors are unimportant with longer training. We stopped after the sweep because no tested configuration had matched the already-confirmed incumbent within the available budget.

6.2.6 Selection

`score_hunt r1250` was chosen as the final DQN tournament candidate on Task 4, the specification’s required gate. It was shortlisted from a 40-round-per-checkpoint curve, confirmed at $N=200$ on an unseen seed, and re-confirmed at $N=1000$ on a second unseen seed. Its three individual paired margins against the three `rule_based_agents` are $+2.290 \pm 0.265$, $+2.281 \pm 0.271$, and $+2.178 \pm 0.267$; every 95% lower bound is above $+1.9$. Checkpoint *C* also passes Task 4 with a smaller positive margin and performs better on the solo and passive-opponent tests reported above, but has a higher crowded-room suicide rate (32.5% against 10.9%).

6.2.7 Controlled comparison with the feature-based agent

Using the shared frozen comparison protocol (`tools/compare_team_agents.py`), both agents were evaluated separately against the same seeded three-`rule_based_agent` opponents at $N=200$ on seed 157. The DQN scored 4.615 ± 0.385 against the feature-based agent’s 3.065 ± 0.320 , with a mean margin over the three opponents of $+1.858 \pm 0.465$ against $+0.233 \pm 0.403$, and was strictly first in 43.0% of rounds against 23.5%. Under this protocol, the DQN passed the required Task 4 gate and the feature-based checkpoint tested at that time did not.

The DQN’s Task 4 result was later confirmed at $N=1000$ on a separate seed, as reported above. The feature-based agent was not re-evaluated at that sample size within the project timeline, so the tournament-entry decision used the $N=200$ controlled comparison. The tested feature-based checkpoint was the checkpoint available for comparison at the time; it was not independently verified as the owner’s final intended submission.

6.3 Head-to-head: Q agent vs. DQN on the same board

Author: Diana Krasnokutskaya

On 16.09 the team compared the two agents by evaluating each one **separately** against three `rule_based_agents`. The DQN (`score_hunt r1250`) scored 4.615 and the Q agent 3.065, and the DQN was selected as the tournament agent. I checked this comparison in two ways before the code deadline.

Which Q model was measured? The comparison script had loaded the Q checkpoint from my commit `c0f36d7`, a copy placed into the DQN branch for a meeting on 15.09, not the current Q model (`f00baeb`, after task-4 training). Replaying `c0f36d7` in the identical protocol ($N = 200$, seed 157) reproduced every metric of the 16.09 run to three decimals, on two machines and two operating systems. This is possible because the seed fixes both the world’s and the agents’ random number generators and the frozen Q policy is a deterministic masked argmax. The 16.09 measurement was correct, but its “Q agent” row described a model three versions old.

Current models, same protocol. Table 10 shows the current Q model re-measured on the same seed as the DQN’s own $N = 1000$ confirmation (seed 223, three `rule_based_agents`).

Table 10: Each agent separately against three `rule_based_agents` ($N = 1000$, seed 223). $\pm$ is the 95% confidence interval half-width.

Agent	Score	Margin vs. mean opponent	Suicide rate
DQN <code>score_hunt r1250</code>	4.961 ± 0.202	$+2.250 \pm 0.238$	10.9%
Q agent <code>f00baeb</code>	5.126 ± 0.213	$+2.482 \pm 0.250$	25.5%

Both agents clearly pass the task-4 gate. The DQN’s lead is gone, although the intervals overlap. The DQN is still the safer policy, suiciding about half as often.

Paired comparison on one board. Evaluating both agents separately compares them against different opponents and different boards. The tournament puts them on the same board, where they compete for the same coins and can kill each other. So I ran both team agents plus two `rule_based_agents` together (four players, `classic`, training off, $N = 1000$, seed 409). Every round is then a paired observation with identical opponents, crates and coins (Figure 5, Table 11).

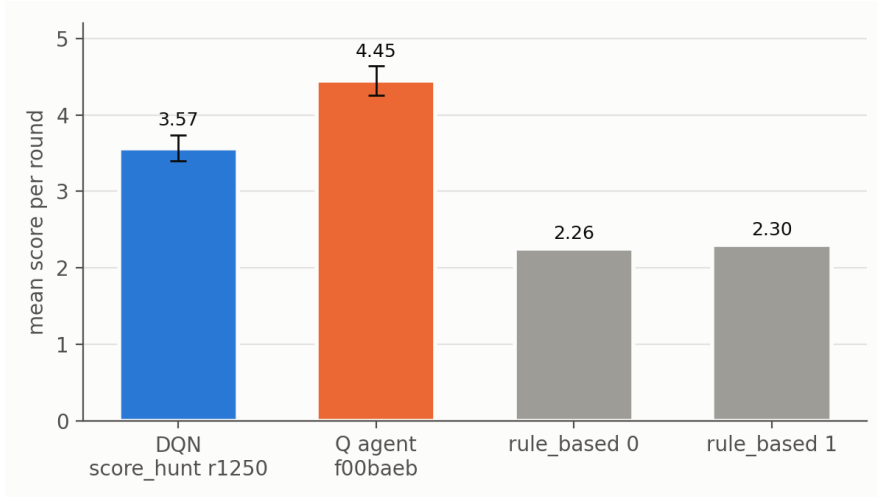


Figure 5: Mean score per round over 1000 four-player rounds (seed 409); error bars are 95% confidence intervals.

Table 11: Head-to-head on one board: both team agents and two `rule_based_agents` ($N = 1000$, seed 409).

Metric	DQN <code>score_hunt r1250</code>	Q agent <code>f00baeb</code>	<code>rule_based 0</code>	<code>rule_based 1</code>
score	3.567 ± 0.17	4.448 ± 0.19	2.257	2.304
coins	2.407	2.988	1.787	1.789
kills	0.232	0.292	0.094	0.103
suicide rate	29.4%	22.2%	39.4%	39.0%
invalid actions	0.859	1.564	5.194	5.269

The paired margin DQN $-$ Q is -0.881 ± 0.286 (95% interval $[-1.167, -0.595]$), so the whole interval lies below zero. The Q agent finished ahead in 52.5% of rounds, the DQN in 37.2%, and 10.3% were ties. Both still beat the rule-based agents (DQN $+1.286 \pm 0.207$, Q $+2.167 \pm 0.226$ against their mean). A pilot with $N = 200$ on a different seed (157) had given -0.870 ± 0.607 . The $N = 1000$ confirmation moved the point estimate by 0.011 and narrowed the interval by a factor of two, which is good evidence that the result is not a seed artefact. A 1-vs-1 of the two

agents alone ($N = 200$) gives -2.690 ± 0.576 and 70.5% of rounds to the Q agent. That setting has more crates and coins per player and no third-party bombs, so we report it only as context, not as a tournament claim.

Notably, the DQN’s suicide rate almost triples (10.9% $\rightarrow$ 29.4%) when the Q agent replaces one rule-based agent. The Q agent apparently creates situations the DQN, fine-tuned only against rule-based opponents, has never seen. This matches the DQN audit in Section 5.2: the `score_hunt` checkpoint is a specialist for exactly one opponent type.

Consequence. The DQN had been selected on the basis of the 16.09 comparison, and that basis no longer holds. The Q agent is ahead or level in every configuration we measured, and it is ahead with a separated confidence interval in the only configuration shaped like the tournament. Two caveats remain. None of our configurations *is* the tournament, where the opponents are other teams’ agents. And the DQN’s lower suicide rate in the solo protocol might matter against opponents that punish mistakes harder than `rule_based_agent` does. Since the tournament ranks by score, we submitted the `q_agent` as our tournament agent. The paired comparison on the same board gave the Q agent a score of 4.448 versus 3.567 for the DQN, with a 95% confidence interval for DQN minus Q of $[-1.167, -0.595]$.

To make this comparison reproducible, I merged both agent branches into `master`, so the comparison is a single command:

```
python tools/evaluate3.py —agents dqn_agent q_agent rule_based_agent
rule_based_agent \ —scenario classic —n-rounds 1000 —seed 409
```

The merge also removed the stale `c0f36d7` copy of the Q agent from the DQN branch, which had caused the mix-up in the first place. The raw per-round results of all five runs are stored under `experiments/runs/` with the prefix `20260917-`.

7 Conclusion

Author: Diana Krasnokutskaya

We trained two structurally different reinforcement learning agents for Bomberman: a linear Q-learning agent over 34 hand-engineered features, and a Double DQN that learns its own features from a six-channel image of the board. Both went through the same curriculum (coins, crates, weak opponents, `rule_based_agent`), and both pass the requirement for the tournament: in frozen play on `classic` against three `rule_based_agents`, each scores clearly more than its opponents (margins of +2.48 and +2.25 over 1000 rounds). On a shared board, however, the Q agent beats the DQN by 0.88 points per round, with a confidence interval that excludes zero, and it finishes ahead in 52.5% versus 37.2% of rounds.

The main findings of the project, as I see them, are:

1. **Domain knowledge was worth more than representational power at our budget.** The linear agent’s features already encode path distances, blast timing and escape routes, so its learning problem is small. The DQN has to discover all of this from image-like input. In three weeks on single CPU threads it learned a safer but less productive policy, and the from-scratch sweep showed that it does not learn bomb safety at all without the curriculum.
2. **Safety is structural, not only learned.** Both agents rely on an action mask that removes provably suicidal actions, and for the Q agent, changing only the mask moved a checkpoint’s margin by 0.9 without retraining. Getting the blast and escape model *exactly* right, down to the step at which an explosion is lethal, was a prerequisite for everything else.
3. **The reward must pay for the event that predicts success.** For the Q agent the most effective single change was rewarding bombs an opponent cannot escape, rather than any bomb near an opponent.

4. **Measurement is the hard part.** Two runs that differed only in their seed ended 1.03 margin points apart, and this was larger than the effect of any hyperparameter we tested. Selecting the best of several short evaluations inflated a candidate’s value from +1.42 to a true +0.55. Evaluating two agents separately instead of on the same board, and with a stale checkpoint, led to the wrong tournament choice until we re-checked. Frozen evaluation, fixed seeds, confidence intervals, a second control run, confirmation on a fresh seed with $N = 1000$, and paired comparisons turned out to matter more than any single modelling trick.

If we had more time, we would work on the following:

- **Train against more varied opponents, including self-play.** A pool of past checkpoints of both agents and other teams’ shared agents would address the DQN’s over-specialisation.
- **Give the DQN the Q agent’s knowledge,** for example the engineered danger and escape features as extra input channels, or a warm start by imitating the Q agent before RL fine-tuning.
- **Reduce the Q agent’s remaining suicides** (22% in the head-to-head). The escape check assumes that opponents stand still; predicting where they can move within the fuse time would close most of the remaining gaps.
- **Run longer DQN experiments with at least three seeds per configuration,** since the sweep’s curves were still rising at 5000 rounds and one seed per arm is below the observed noise level.

For next year’s game setup, we would suggest:

- Report per-agent statistics in the round summary. The framework currently sums coins, kills and suicides over all agents, so from task 3 on every team has to write its own evaluation script.
- Document exactly at which steps a bomb makes a tile lethal. We had to reverse-engineer this from `environment.do_step`, and an off-by-one error there directly causes suicides.
- Provide a reference evaluation script with seeds and confidence intervals, which would make results comparable across teams.

Work distribution

The team followed the component-based split agreed at the kickoff rather than assigning one model to each person. All three members contributed to the curriculum, bomb-safety decisions, evaluation protocol, experiment discussions, and report review. The main areas of responsibility were as follows.

References

- [1] V. Mnih, K. Kavukcuoglu, D. Silver, et al. *Human-level control through deep reinforcement learning*. Nature, 518(7540):529–533, 2015.
- [2] H. van Hasselt, A. Guez, and D. Silver. *Deep Reinforcement Learning with Double Q-learning*. Proc. AAAI Conference on Artificial Intelligence, 2016.

Work package	Main contribution	Lead
WP1	Environment interface, BFS features, blast danger, escape checks, and action masking for the feature-based agent	Daniela
WP2	Linear Q-learning agent, reward shaping, curriculum training, and Q-agent checkpoint selection	Daniela
WP3	DQN architecture, Double DQN training loop, replay buffer, reward profiles, and DQN training experiments	Luis
WP4	Evaluation harness, experiment logging, plots, frozen comparisons, and submission checks	Luis
WP5	Task-3 opponent analysis, hyperparameter sweeps and ablations for both models, head-to-head evaluation, interpretation of results, and report synthesis	Diana

- [3] A. Y. Ng, D. Harada, and S. Russell. *Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping*. Proc. 16th International Conference on Machine Learning (ICML), 1999.
- [4] F. Pardo, A. Tavakoli, V. Levnik, and P. Kormushev. *Time Limits in Reinforcement Learning*. Proc. 35th International Conference on Machine Learning (ICML), 2018.
- [5] U. Köthe. *Machine Learning Essentials: Reinforcement Learning, Parts 1 and 2*. Lecture, Heidelberg University, 14–16 July 2026.
- [6] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. 2nd edition, MIT Press, 2018.
- [7] R. S. Sutton. *Learning to Predict by the Methods of Temporal Differences*. Machine Learning, 3(1):9–44, 1988.
- [8] C. J. C. H. Watkins and P. Dayan. *Q-learning*. Machine Learning, 8(3–4):279–292, 1992.
- [9] G. A. Rummery and M. Niranjan. *On-line Q-learning Using Connectionist Systems*. Technical Report CUED/F-INFENG/TR 166, Cambridge University, 1994.
- [10] S. Singh, T. Jaakkola, M. L. Littman, and C. Szepesvári. *Convergence Results for Single-Step On-Policy Reinforcement-Learning Algorithms*. Machine Learning, 38(3):287–308, 2000.
- [11] H. van Hasselt, Y. Doron, F. Strub, et al. *Deep Reinforcement Learning and the Deadly Triad*. arXiv:1812.02648, 2018.
- [12] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. *Proximal Policy Optimization Algorithms*. arXiv:1707.06347, 2017.
- [13] C. B. Browne, E. Powley, D. Whitehouse, et al. *A Survey of Monte Carlo Tree Search Methods*. IEEE Transactions on Computational Intelligence and AI in Games, 4(1):1–43, 2012.
- [14] S. Ross, G. J. Gordon, and J. A. Bagnell. *A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning*. Proc. 14th International Conference on Artificial Intelligence and Statistics (AISTATS), 2011.